SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

Name: K. Dedeepya

Reg no: 192324252

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.

Field	Analysis
HTTP/1.1 200 OK	The status line is syntactically valid.
Date	The weekday and date do not match. April 1, 2026, is a Wednesday, not a Monday.
Content-Type: text/html	Valid. It indicates that the response contains HTML content.
Content-Length: -150	Invalid because the content length cannot be negative.
Connection: keep-alive	Valid for HTTP/1.1, although persistent connections are already the default unless closed.

2. Analyse the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the response body in bytes. A negative value such as -150 is invalid.

Impact:

- The response may be rejected or treated as a protocol error.
- The receiver cannot rely on this value to determine the response body length.
- Incorrect message framing may cause communication problems.

3. Determine whether the status code matches the response correctness.

200 OK indicates that the request was successfully processed. However, it does not guarantee that every response header is valid. Therefore, the status line indicates success, but the response contains an invalid Content-Length value and an incorrect date weekday.

4. Suggest corrections for the identified errors.

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Note: The value 150 is correct only if the actual response body contains exactly 150 bytes.

Conclusion: Correct HTTP headers are essential for reliable communication between browsers and servers.

2: DOM Structure Analysis

Consider the following DOM tree representation:

<html>

<head>

<title>TestPage</title>

</head>

<body>

<p>HelloWorld

</body>

</html>

Questions:

1. Identify missing or improperly closed elements.

The paragraph element <p> is not explicitly closed before the closing tag. The other displayed elements have closing tags.

2. Analyse how the browser will interpret this structure.

The browser's HTML parser attempts to recover from common markup errors. In this example, it normally closes the paragraph automatically before closing the div.

The browser can therefore construct a usable DOM even though the source code needs correction.

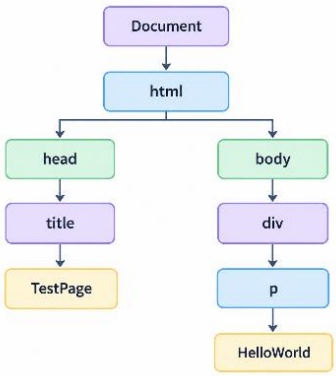
3. Determine the impact on rendering and layout.

- The text HelloWorld will generally appear inside the paragraph.
- The browser may automatically correct the missing closing tag.
- Incorrect nesting in more complex documents can cause unexpected layouts and DOM structures.
- Scripts and accessibility tools may be affected when the actual DOM differs from the intended structure.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>TestPage</title>
</head>
<body>
   <div>
      <p>HelloWorld</p>
   </div>
</body>
</html>
```

DOM Tree Diagram



For Interpretation: The DOM represents a webpage as a tree of nodes. Each element has a parent-child relationship, allowing browsers and JavaScript to access and manipulate webpage content.

Conclusion: Correct element nesting improves the readability, accessibility, and maintainability of HTML documents.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Suitability
GET	/login?user=admin&pass=1234	Credentials appear in the URL	Not suitable for passwords
POST	/login	Form data is sent in the request body	Preferred for login submission

Questions:

1. Interpret the differences in data transmission between GET and POST.

GET: Retrieves information from a server. Parameters are commonly included in the URL.

Example:

GET /login?user=admin&pass=1234 HTTP/1.1

Host: example.com

POST: Submits data to a server. Form data is generally included in the request body.

Example:

POST /login HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

2. Analyze which method is more suitable for login and why.

POST is more suitable for login authentication because credentials are not placed directly in the URL.

It reduces the risk of exposing passwords through browser history, copied URLs, and URL-based logs. However, POST alone does not encrypt the data.

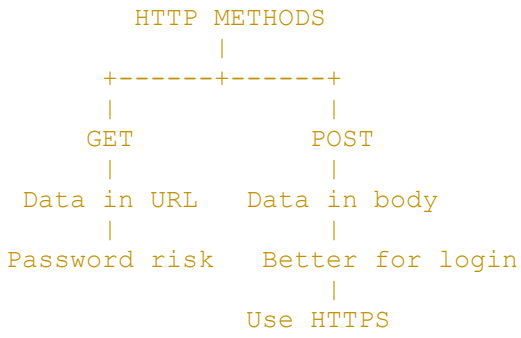
3. Identify security risks in using GET for authentication.

1. Passwords may appear in browser history.
2. URLs containing credentials may be stored in server logs.
3. Users may accidentally share sensitive URLs.
4. Credentials may be exposed through other systems that record URLs.

4. Suggest best practices for secure login implementation.

- Use HTTPS to encrypt data during transmission.
- Submit login credentials using POST.
- Store passwords using secure password-hashing algorithms such as Argon2id or bcrypt.
- Validate credentials on the server.
- Use secure session cookies with suitable security attributes.
- Apply rate limiting to reduce automated login attempts.

GET vs POST Diagram



Conclusion: POST is preferable for login forms, but secure authentication also requires HTTPS, safe password storage, and proper session management.

4: HTML Code Inspection

Analyze the following HTML snippet:

<!DOCTYPE html>

```
<html>

<head>

<title>Sample</title>

</head>

<body>

<h1>Welcome

<p>This is a paragraph

<img src="image.jpg">

</body>

</html>
```

Questions:

1. Identify missing closing tags.

The <h1> and elements do not have explicit closing tags. They should be closed using </h1> and .

2. Analyze issues with the <img> tag.

The image element is: **<img src="image.jpg">**

It has two considerations:

Missing alt attribute: Alternative text improves accessibility and can describe the image if it fails to load.

Image path: The file must exist at the specified location for the image to appear.

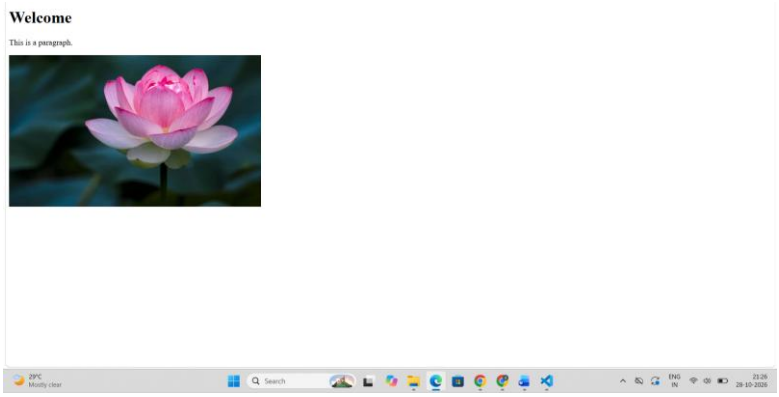
In HTML5, <img> is a void element and does not require a closing tag.

3. Determine how browsers will handle these errors.

- The browser normally recovers from missing heading and paragraph closing tags.
- The heading and paragraph may still appear correctly.
- If the image file is missing, the browser may display a broken-image indicator.
- Missing alternative text can reduce accessibility for users relying on screen readers.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sample</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
    <img src="C:/Users/Dedeepya/Downloads/image.jpeg" width="500" height="300">
</body>
</html>
```



5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs

3. HTTP request is sent

4. Server processes request

5. Response is returned

6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

Step 1: User enters a URL: The browser identifies the website address and requested resource.

Step 2: DNS lookup: The Domain Name System (DNS) translates the domain name into an IP address. A cached result may eliminate the need for a new lookup.

Step 3: HTTP request is sent: The browser establishes the necessary connection and sends an HTTP request to the server. HTTPS also provides TLS protection.

Step 4: Server processes the request: The server locates the requested resource, executes application logic if needed, and may retrieve data from a database.

Step 5: Response is returned: The server sends an HTTP response containing a status code, headers, and, where applicable, a response body.

Step 6: Browser renders the page: The browser processes HTML, CSS, and JavaScript to display the webpage. It may request additional resources such as images and stylesheets.

2. Identify where delays can occur.

Stage	Possible Delay
DNS lookup	Slow DNS response or cache miss
Connection setup	Network latency
Server processing	Heavy workload or slow database queries
Response transfer	Large files or low bandwidth
Browser rendering	Large images or complex JavaScript

3. Analyse the role of DNS in the process.

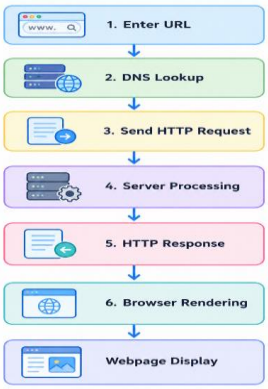
DNS converts human-readable domain names into IP addresses so that browsers can locate servers.

Its main benefits are:

- Simplifies website access.
- Helps locate the destination server.
- Uses caching to reduce repeated lookups.

Slow DNS resolution can delay connection establishment. However, DNS does not normally transfer the webpage itself.

4. Suggest optimizations to improve performance.



Conclusion: Efficient DNS resolution, server processing, and resource delivery reduce loading delays and improve the user experience.